

配置总表.xlsx 阅读指南

导出/导入命令：`npm run config:export-xlsx` / `npm run config:import-xlsx`
文件位置：`交付/配置总表.xlsx`（已在 `.gitignore` 中忽略，属于本地编辑产物，不进 Git）
导入是**全或无**：任一域校验失败，15 个目标 JSON 一个都不写。失败时会打印出错的配置 `path`。

一、四种表结构

19 张工作表分四类，每类的行列含义不同。

结构 A：path/value 长表（8 张）

`combat`、`waves`、`enemies`、`difficulty`、`progression`、`economy`、`bounty`、`input`、以及 `texts.global`

固定 4 列：

列	含义
<code>path</code>	JSON Pointer 风格路径。 <code>#</code> 是根， <code>#/canvas/width</code> 对应 <code>{canvas:{width:...}}</code> ， <code>#/list/0</code> 是数组第 0 项
<code>label</code>	中文显示名，来自 <code>src/editor/labels.ts</code> 单源。 只读 ，改了不影响导入
<code>value</code>	唯一要编辑的列 。容器行（object/array）留空
<code>type</code>	<code>object</code> / <code>array</code> / <code>number</code> / <code>string</code> / <code>boolean</code> 。容器行在此声明结构

规则：

- 容器行（`type=object/array`）**不能删**，它定义结构和键顺序。
- 数组顺序由 `path` 里的下标决定，不是由行序决定。要重排数组，改下标。
- 前 4 行是表头：第 1 行标题、第 2 行用法提示、第 3 行技术列名（隐藏用途，别删）、第 4 行中文表头。数据从第 5 行开始。

结构 B：一行一实体（5 张）

`gods`、`relics`、`evolutionRecipes`、`waveRewards`、`tuner`

每行一个配置实体，每列一个字段。对象/数组字段用 JSON 文本填在单元格里。

末尾三列 `__fieldorder` / `__fieldTypes` / `__textNodes` 是**技术列**，用于导入时还原字段顺序与类型。不要编辑，不要删。

结构 C：skills 四张关联表

技能配置太深，拆成四张表用 `cardid` / `star` / `mode` / `bindingIndex` 做联合主键串联。

结构 D：_meta

`formatVersion`（当前 1）与 `textsEntityOrder`（文案实体的原始顺序，保证 round-trip 零 diff）。不要动。

二、逐表说明

combat — 战斗基础

画布尺寸、炮台坐标、心防（hp）上下限、基础伤害/射速/射程/同发数、子弹速度、碰撞判定等全局战斗常量。改这里等于改所有局的底盘。

waves — 波次与刷怪

波数、每波敌人数量基数与递增、刷怪间隔曲线（`base` / `perWave` / `min`）、首刷延迟、Boss 波位置、`spawnMode`（`interval` 定时 / `budget` 预算）。这是难度曲线的第一来源。

enemies — 敌人原型

各敌人类型的血量、速度、伤害、体型、抗性（冰冻/击退）、掉落权重。

difficulty — 多难度系统

难度档位对 waves/enemies 的乘数与覆盖项。

progression — 经验与升级

XP 曲线、每级奖励、遗物化经验升级（C4）相关参数。

economy — 经济

金币/资源产出与消耗、商店定价、掉落价值。对应 docs/S4a_经济拍板_provisional.md。

bounty — 精英悬赏

Bounty 触发条件、精英词条、奖励规格、流派倾向权重。

input — 输入层

拖放阈值、点击判定、触控校准参数（对应 docs/T1_触控校准记录.md）。行数很少（10 行）。

gods — 五神

一行一个神。

列	含义
id	神 id (storm / frost / fire / ...)
textKey	指向 texts.gods.<id>
anchorCardIds	锚点卡 id 数组 (JSON)。填不存在的卡 id 会被导入校验拦截
variableCardIds	可变卡 id 数组 (JSON)
mainRosterSize / subRosterSize	主池 / 副池规模
text:#/name、text:#/theme	该神的名称与主题文案

relics — 遗物

列	含义
id / god / rarity	标识、归属神、稀有度
targetTags	生效的协同标签数组 (JSON)
effects	效果定义 (JSON)
poolInfluence	对神池抽取权重的影响
maxStacks	最大叠加层数
text:#/name、text:#/desc	遗物名与描述

evolutionRecipes — 卡间进化配方（C6）

ingredientA + ingredientB → outputCardId（outputStar 星）。allowedPhase 限制可触发的阶段。

waveRewards — 波末基础奖励（C2/C2R）

列	含义
<code>section</code>	<code>floor</code> = 保底必给； <code>choice</code> = 进入三选一候选池
<code>id</code>	奖励 id
<code>stat</code>	加成属性（heal / damageAdd / fireRateAdd / maxHpAdd / rangeAdd / xpGainPct）
<code>add</code>	加值。注意百分比类用小数（0.15 = +15%）

tuner — 调参面板定义

定义调参器 UI 里暴露哪些参数、怎么呈现。它本身不是数值，是「哪些数值可以被面板改」的元配置。

列	含义
<code>path</code>	指向真实配置的点号路径，如 <code>waves.spawnInterval.base</code>
<code>type</code>	<code>number</code> / <code>boolean</code> / <code>enum</code> / <code>text</code>
<code>labelKey</code>	指向 <code>texts.tuner.params.*</code> 的文案键
<code>group</code>	面板分组（waves / bounty / ...）
<code>applyPolicy</code>	<code>immediate</code> 立即生效 / <code>waveDeferred</code> 下一波生效
<code>options</code>	enum 类型的可选值数组（JSON）
<code>min</code> / <code>max</code> / <code>step</code>	number 类型的滑杆范围
<code>exposed</code>	是否在面板显示

三、skills 四张关联表（重点）

技能数据是「卡 → 星级形态 → 绑定 → 效果 → 参数」的五层树，拆成四张平表。

1. `skills.cards` — 一行一张卡（41 张）

左半区是卡的基本盘：

列	含义
<code>id</code>	卡 id, 全表关联主键
<code>god</code>	归属神
<code>category</code>	projectile / control / domain / defense / utility
<code>synergyTags</code>	协同标签数组 (JSON), 遗物与流派靠它匹配
<code>textKey</code>	指向 <code>texts.cards.<id></code>
<code>teaching</code>	是否教学卡
<code>implementationBatch</code>	实现批次
<code>recipeOnly</code>	是否只能由配方产出 (不进普通掉落池)
<code>starsMeta</code>	各星级的形态标记 (3=core / 5=dual / 6=transform)。 注意: 这里只有元数据, 效果不在这, 在 bindings/effects 表
<code>amplifyAxis</code>	4★ 公共放大轴的描述与参数增量
<code>consumableMeta</code>	消耗态投放方式 (point 等)、插值方式、各档锚点半径
<code>affixPool</code>	随机数值槽候选池 (C7): 抽几条、候选 stat 与权重、min/max/step
<code>evolutionTree</code>	单卡进化树 (C5): 3★/5★ 分支选项及其效果覆盖
<code>fusionPolicy</code>	装备被动融合策略
<code>designNotes</code>	设计备注, 纯注释, 不进游戏

右半区 (第 P 列起) 是文案, 23 列:

- `text:#/name` — 卡名
- `text:#/hand/shortByTier/{1,3,6}` — 手牌态各档一句话描述
- `text:#/hand/milestones/{3,6}/{title,detail,fx}` — 手牌态升星里程碑标题/详情/特效等级
- `text:#/equip/shortByTier/{3,5,6}` — 装备态各档一句话
- `text:#/equip/milestones/{3,5,6}/{title,detail,fx}` — 装备态里程碑
- `text:#/overview` — 卡牌总览文案

`fx` 值为特效等级: `core` / `major` / `transform`。

2. skills.bindings — 触发绑定

列	含义
<code>cardId</code>	卡 id
<code>star</code>	equip 模式用 3/5/6; consume 模式用 1/3/6
<code>mode</code>	<code>equip</code> 装备态 (常驻被动) / <code>consume</code> 消耗态 (拖放释放)
<code>bindingIndex</code>	同一 (卡, 星, 模式) 下的第几条绑定, 从 0 开始
<code>trigger</code>	触发器: <code>onHit</code> / <code>onFire</code> / <code>onKill</code> / <code>interval</code> 等。consume 模式为空 (即时释放)
<code>triggerParams</code>	触发器参数 JSON, 如 <code>{"requiresSource": "chain"}</code> 、 <code>{"seconds": 1.2}</code>

例: `chainLightning` / 5 / equip 有两条绑定 —— index 0 是 onHit 连锁, index 1 是 onKill 且要求击杀来源为 chain (即「断链重引」)。

3. skills.effects — 效果原子

列	含义
<code>cardId</code> / <code>star</code> / <code>mode</code> / <code>bindingIndex</code>	关联回 bindings 表的同名四列
<code>effectIndex</code>	该绑定下第几个效果，从 0 开始
<code>parentEffectPath</code>	空 = 顶层效果；0 = 挂在父效果 0 的 <code>params.effects</code> 里；0.1 = 再往下钻到父效果 0 的子效果 1
<code>atom</code>	效果原子名： <code>chain</code> / <code>slow</code> / <code>stun</code> / <code>burn</code> / <code>knockback</code> 等，取值受 <code>ATOM_CONTRACT</code> 约束
<code>__hasParams</code> / <code>__paramOrder</code>	技术列，不要动

4. `skills.effectParams` — 效果参数 (1325 行)

列	含义
前 6 列	与 effects 表完全对应，用于定位是哪一个效果
<code>paramName</code>	参数名，如 <code>bounces</code> / <code>damageRetention</code> / <code>searchRange</code> / <code>ratio</code> / <code>duration</code>
<code>paramLabel</code>	中文名，来自 <code>ATOM_CONTRACT</code> ，只读
<code>value</code>	参数值。number/int/boolean 是 Excel 原生类型；record 或联合对象是 JSON 文本
<code>__atom</code>	所属原子，只读，方便筛选

清空 `value` = 删除该参数键，运行时改用 `ATOM_CONTRACT` 的默认值。这是有意设计，不是 bug。

改一个技能数值的完整路径

例：把 3★ 装备态连环闪电的弹跳次数从 2 改成 3。

在 `skills.effectParams` 筛选 `cardId=chainLightning`、`star=3`、`mode=equip`、`bindingIndex=0`、`effectIndex=0`、`parentEffectPath` 为空、`paramName=bounces`，改 `value` 列。其余三张表都不用碰。

四、文案在哪

文案类型	存放位置
卡牌文案	<code>skills.cards</code> 的 <code>text:#/...</code> 列 (P 列起，共 23 列)
神祇文案	<code>gods</code> 的 <code>text:#/name</code> 、 <code>text:#/theme</code>
遗物文案	<code>relics</code> 的 <code>text:#/name</code> 、 <code>text:#/desc</code>
其余全部	<code>texts.global</code>

`texts.global` 按顶层节点分布 (共 2033 行)：

节点	行数	内容
<code>#/evolution</code>	1569	进化分支文案：每个分支的 name / summary / intent / keywords / buildFit。占了 77%
<code>#/tuner</code>	194	调参面板的分组名与参数标签
<code>#/effectText</code>	63	效果与触发器的动态描述模板
<code>#/glossary</code>	34	术语表（穿透、连锁...）
<code>#/toast</code>	33	飘字提示，支持 <code>{wave}</code> 等占位符
<code>#/affixes /</code> <code>#/affixHelp</code>	42	词条名与说明
<code>#/decisions</code>	19	决策队列（神池抽取等）文案
<code>#/intermission</code>	17	波间阶段
<code>#/result</code>	14	结算页
<code>#/cards</code>	12	只有 <code>#/cards/fusion</code> 这类无实体归属的残余节点
其余	~30	按钮、通道名、中央提示、万能卡、升级、波末奖励属性名

写回时各自回到 `src/data/texts.json` 的对应位置。只改遗物名的话，`src/config/base/*.json` 应该零 diff。

五、通用注意事项

- 第 3 行技术表头不要删也不要改，导入靠它定位列。第 4 行的中文表头才是给人看的。
- ☐ 开头的列全部是技术列，不要编辑。
- 只改 value 列 / 数据列，label、paramLabel 是从代码单源生成的，改了不生效。
- 悬空 id（例如 `gods.anchorCardIds` 里写不存在的卡）会被导入校验拦截并报出 `$.gods.gods[0].anchorCardIds`。
- 改完的验证顺序：`npm run config:import-xlsx` → `git diff` 确认改动范围符合预期 → `npm run validate` → `npm run test`。
- 干净 round-trip（导出后立刻导入）应该零 diff。如果不是零 diff，说明工具有问题，先别改数据。